

WI-FI PERFORMANCE REPORT

Capitani Federico, de Pietro Domenico, Oliverio Riccardo, Rigotti Ludovico

Politecnico di Torino

1 INTRODUCTION

The network speed perceived by applications and users is usually lower than the nominal rate reported by the network interface. This difference is due to the bandwidth consumed by protocol headers, frame overhead, and, in wireless communications, by the medium-access mechanisms introduced by CSMA/CA.

In this report, we evaluate the application-layer goodput of different local network scenarios using `iperf3`. The experiments are performed over IEEE 802.11g standard. We vary the TCP interface MTU and the UDP application payload size in order to observe the impact of header overhead, fragmentation, and wireless contention on goodput.

The final goal is to compare the measured results with theoretical expectations and identify the main factors that limit the achievable goodput in wired and wireless scenarios.

2 TOOLS AND THEORY

2.1 Measurement tool

Table 1: Summary of the commands used in the experiments.

Protocol	Command template
UDP	<code>iperf3 -c <server> -t 10 -J -u -l <L> -b -R</code>
TCP	<code>iperf3 -c <server> -t 10 -J -R</code>
TCP MTU	<code>ip link set <iface> mtu <MTU></code>

We used `iperf3` in client-server mode to measure application-layer goodput. UDP tests varied the payload size with `-l` and sending-rate limit with `-b` (unlimited in case of Ethernet to Ethernet and limited at 40Mbps in case of the other scenarios); TCP tests varied the interface MTU to control the MSS. Each run lasted 10 seconds and was repeated ten times, collecting receiver-side goodput `-R` from the JSON output. We also used Wireshark for graphical traffic analysis and an additional Python script to generate the final plots from the collected CSV files.

2.2 Theoretical maximum goodput

To determine the Theoretical Maximum Throughput (TMT) at the application level, we decompose the formula presented in [1]:

$$TMT_{APP} = \frac{\beta}{\alpha + \beta} \cdot \frac{8 \cdot MSDU}{a \cdot MSDU + b} \cdot 10^6 \quad (1)$$

Where TMT_{APP} represents the theoretical goodput at the application level, which is the final metric used for comparison with empirical data.

The rightmost part of the equation corresponds to $TMT_{802.11}$ (or MAC TMT), which is the maximum throughput at the Wi-Fi MAC layer. It accounts for channel access mechanisms (e.g., CSMA/CA, ACKs, backoff) and is defined as:

$$TMT_{802.11} = \frac{8 \cdot MSDU}{a \cdot MSDU + b} \cdot 10^6 \quad (2)$$

The variable β denotes the application payload size:

- For UDP: $\beta = L$
- For TCP: $\beta = MTU - 40$ (accounting for headers)

The variable α represents the overhead above the MAC layer, including all non-application headers that must be transmitted:

- For UDP: $\alpha = 20_{IP} + 8_{UDP} + 8_{LLC} = 36$ B
- For TCP: $\alpha = 20_{IP} + 20_{TCP} + 8_{LLC} = 48$ B

Finally, the $MSDU$ (MAC Service Data Unit) represents the total data payload delivered to the Wi-Fi MAC layer. It is calculated as:

- For UDP: $MSDU = L + 36$
- For TCP: $MSDU = MTU + 8$

For IEEE 802.11g, we estimate the expected goodput using the theoretical maximum throughput (TMT) model proposed by [1]. Although the OFDM PHY parameters are primarily defined in the context of IEEE 802.11a, we use the 54 Mbps CSMA/CA parameters as a reasonable approximation for our IEEE 802.11g experiments, given that 802.11g adopts OFDM for its higher data rates.

Furthermore, our model does not account for the RTS/CTS mechanism, as this feature was not explicitly enabled during the measurements. The parameters a and b are extracted from [1], where a represents the time component that scales with the packet size, and b represents the fixed time overhead. Specifically, we set $a = 0.14815$ and $b = 159.94$.

Table 2: Maximum theoretical application goodput for IEEE 802.11g.

Protocol	Scenario	Best size (B)	TMT_{APP} (Mb/s)
TCP	single-hop	MTU 1500	30.468
TCP	Wi-Fi-to-Wi-Fi	MTU 1500	15.234
UDP, no frag.	single-hop	$L = 1472$	30.719
UDP, no frag.	Wi-Fi-to-Wi-Fi	$L = 1472$	15.359
UDP, frag.	single-hop	$L = 2944$	39.160
UDP, frag.	Wi-Fi-to-Wi-Fi	$L = 2944$	19.580

Table 2 reports the maximum theoretical goodput values used as reference for the experimental comparison. For TCP, the maximum value is obtained with MTU 1500, because the MSS is maximized and the fixed IP/TCP/LLC overhead is amortized over the largest useful payload. For UDP, $L = 1472$ B is the largest non-fragmented payload with a standard 1500 B MTU. The fragmented case at $L = 2944$ B gives a higher theoretical value because the model only accounts for payload efficiency and MAC airtime, but in real measurements it can be worse: each UDP datagram is split into IP fragments, and the loss of one fragment causes the loss of the entire datagram.

3 LAB SETUP AND SCENARIOS

The scenarios are organized from the simplest wired baseline to more complex wireless configurations, in order to isolate the effect of wireless contention, hop count, and channel conditions.

3.1 Measurement Setup

Table 3 and table 4 presents, respectively, the physical and wireless setup and network configuration adopted across all different scenarios.

Table 3: Physical and wireless setup.

Parameter	Value / Description
Device	NETGEAR DGN1000
Ethernet link	100 Mb/s, full-duplex
Wi-Fi band	2.4 GHz
Channel	11
Wi-Fi standard	IEEE 802.11g
Security	WPA2-Personal (AES)
AP-client distance	≈1.5 m, line-of-sight
Measurement tool	iperf3 v3.18

Table 4: Network configuration.

Device	IP address	Subnet	Configuration	OS
Access point	192.168.0.1	/24	Gateway / DHCP server	
Client host	192.168.0.2	/24	DHCP or static	Linux
Server host	192.168.0.3	/24	DHCP or static	Linux

3.2 Traffic Configuration

For each scenario, both TCP and UDP tests are performed using iperf3. For TCP, the interface MTU is varied in order to study the effect of segment size and header amortization. For UDP, the application payload size is varied in order to observe the impact of payload efficiency and IP fragmentation. Table 5 lists all the values of the MTU sizes and UDP payload sizes used during measurement tests.

Table 5: Traffic parameters used in the experiments.

Parameter	Values
TCP MTU sizes	{128, 256, 512, 768, 1024, 1280, 1500} B
UDP payload sizes	{16, 64, 128, 256, 512, 1024, 1472, 1473, 2048, 2944} B

3.3 Experiment Scenarios

The experiments consider three scenarios, summarized in Table 6. Scenario S1 is used as the wired baseline, where both endpoints are connected via Ethernet. Scenario S2 represents a single-hop wireless configuration, where the server is connected via Ethernet and the client is connected via Wi-Fi. Scenario S3 represents a Wi-Fi to Wi-Fi configuration, where traffic crosses the wireless medium twice: once from the client to the access point and once from the access point to the server. This organization allows the measurements to move from the simplest case to progressively more complex wireless configurations. In this way, the impact of wireless medium access and the additional Wi-Fi hop can be observed separately in the results.

Table 6: Scenario definitions.

ID	Client	Server	Wireless hops
S1	Ethernet	Ethernet	0
S2	Wi-fi	Ethernet	1
S3	Wi-Fi	Wi-Fi	2

Finally, TCP and UDP are evaluated separately in order to observe how MTU, payload size, protocol overhead, and fragmentation affect the measured application-layer goodput.

4 RESULTS

4.1 Ethernet to Ethernet scenario

The plot in Figure 1 shows, in case of TCP protocol, how the goodput varies as a function of the MTU size, while the plot in Figure 2 shows, in case of UDP protocol, how the goodput varies as a function of the frame size.

The results for the Ethernet to Ethernet scenario at 100 Mbps nominal speed demonstrate high stability and close alignment with theoretical efficiency limits. For TCP protocol, the maximum average goodput reached 94.05 Mbps (min: 94.05, max: 94.06, σ : 0.003 Mbps) using MTU of 1500-byte, matching the predicted efficiency of μ TCP ≈ 0.949 .



Figure 1: Ethernet to Ethernet - TCP protocol.

UDP achieved a peak average goodput of 95.79 Mbps (min: 95.75, max: 95.81, σ : 0.018 Mbps) at the 1472-byte MTU limit, consistent with μ UDP ≈ 0.957 . A performance drop to 90.85 Mbps was observed at 1473 bytes due to IP fragmentation overhead, which later recovered to an average of 95.68 Mbps for 2944-byte payloads as the payload-to-header ratio improved.

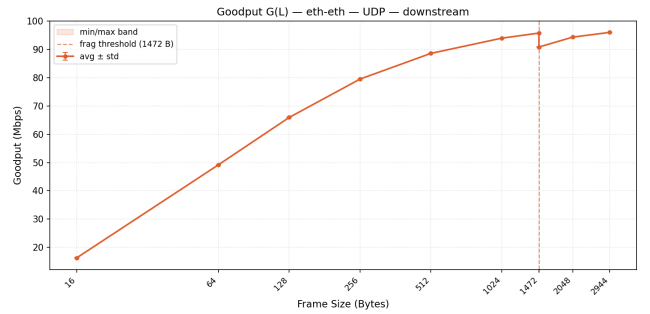


Figure 2: Ethernet to Ethernet - UDP protocol.

Small frame sizes (e.g., 128-byte TCP at 72.97 Mbps and 16-byte UDP at 16.30 Mbps) significantly reduced efficiency due to the high relative cost of headers. Across all tests, standard deviations remained below 0.07 Mbps, indicating a congestion-free environment. All measured goodput values G strictly satisfied the theoretical bound $G < \mu C$, where $C=100$ Mbps. These results validate the impact of protocol encapsulation and fragmentation on effective application throughput in a wired environment.

The values of the goodput for each MTU size or datagram payload size chosen for the tests are reported in the following Table 7 and Table 8.

Table 7: Average goodput for each MTU size - TCP protocol.

MTU size	Goodput
256	72.9748
512	83.5635
768	88.8063
1024	91.4286
1280	93.1067
1500	94.0499

Table 8: Average goodput for each datagram payload size - UDP protocol.

Datagram size	Goodput
16	16.296
64	49.1921
128	65.9626
256	79.53
512	88.6148
1024	93.9881
1472	95.7891
1473	90.8264
2048	94.3517
2944	95.6839

4.2 Ethernet to Wi-Fi scenario

The plot in Figure 3 shows, in case of TCP protocol, how the goodput varies as a function of the MTU size, while the plot in Figure 4 shows, in case of UDP protocol, how the goodput varies as a function of the frame size.

In the Ethernet-to-Wi-Fi scenario, the 802.11g link (54 Mbps nominal speed) acts as the system bottleneck. TCP achieved a peak average goodput of 21.59 Mbps (min: 21.07, max: 22.02, σ : 0.28 Mbps) using MTU of 1500-byte.

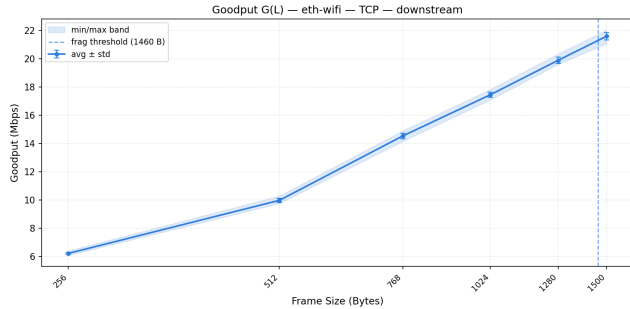


Figure 3: Ethernet to Wi-Fi - TCP protocol.

In contrast, UDP performance was severely degraded, peaking at only 9.55 Mbps (min: 5.19, max: 11.45, σ : 2.55 Mbps) at 1472-byte.

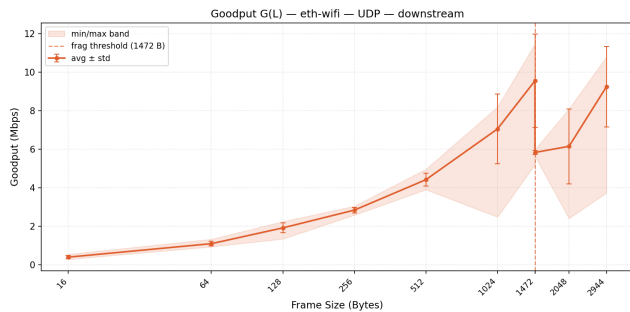


Figure 4: Ethernet to Wi-Fi - UDP protocol.

The significant underperformance of UDP relative to TCP stems from the lack of flow control: the 100 Mbps Ethernet source overwhelms the Access Point's buffer, leading to massive tail-drops and reduced effective goodput. Conversely, TCP's congestion control limits the transmission rate to the actual bottleneck capacity, maintaining higher stability and efficiency. The deviation from the ideal theoretical ≈ 27 Mbps limit suggests a PHY rate down-shift due to environmental interference or distance. UDP fragmentation at 1473-byte caused a sharp drop to 5.83 Mbps due to double-header overhead and increased packet loss probability. High standard deviations in UDP measurements highlight the instability of the wireless buffer under saturation. All results strictly satisfy the theoretical bound $G < \mu C_{\text{bottleneck}}$.

The values of the goodput for each MTU size or datagram payload size chosen for the tests are reported in the following Table 9 and Table 10.

Table 9: Average goodput for each MTU size - TCP protocol.

MTU size	Goodput
256	6.2174
512	9.9709
768	14.5317
1024	17.4466
1280	19.8895
1500	21.5879

Table 10: Average goodput for each datagram payload size - UDP protocol.

Datagram size	Goodput
16	0.3938
64	1.0929
128	1.9135
256	2.8365
512	4.4165
1024	7.0499
1472	9.5477
1473	5.8292
2048	6.1471
2944	9.2429

4.3 Wi-Fi to Wi-Fi scenario

The Wi-Fi to Wi-Fi double-hop scenario demonstrates severe performance degradation for both TCP and UDP compared to wired baselines, primarily because all traffic must access the wireless medium twice. Both protocols are significantly penalized by CS-MA/CA contention mechanisms, whose impairments become multiplicative across multiple wireless hops. Furthermore, both protocols exhibit minimal efficiency at the smallest frame sizes, as the fixed protocol header overhead consumes a disproportionate amount of the available bandwidth relative to the useful application payload. For the TCP protocol, the measured application goodput scales monotonically with the frame size, reaching a maximum average of approximately 11.5 Mbps at an MTU of 1500 bytes. This peak performance remains substantially below the theoretical maximum of 15.234 Mbps predicted for a double-hop configuration. While larger MTUs generally improve goodput by amortizing the per-frame header cost, these performance gains begin to diminish above 1280 bytes on wireless links, where the transmission of larger frames incurs higher collision penalties.

For the UDP protocol, application goodput peaks at approximately 13.3 Mbps at the 1472-byte payload limit, falling short of the 15.359 Mbps non-fragmented theoretical maximum. The dominant performance factor is the IP fragmentation boundary at 1472 bytes: immediately beyond this threshold, goodput experiences

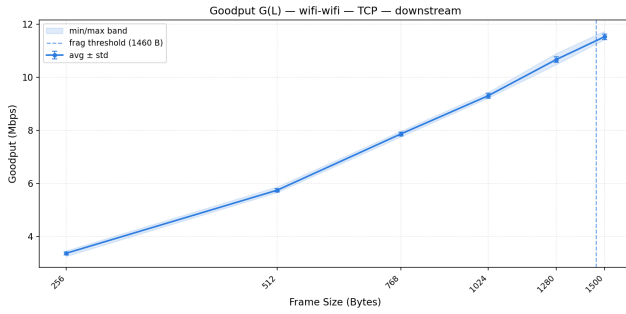


Figure 5: Wi-Fi to Wi-Fi - TCP protocol.

a sharp, unrecoverable collapse to roughly 7.2 Mbps. This post-fragmentation zone introduces massive variance and extreme performance penalties because elevated frame-loss probabilities in the noisy double-hop environment compound the fragment discard probability; the loss of a single IP fragment across either wireless link causes the loss of the entire UDP datagram.

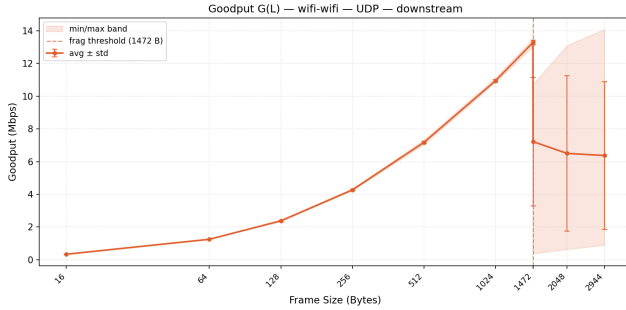


Figure 6: Wi-Fi to Wi-Fi - UDP protocol.

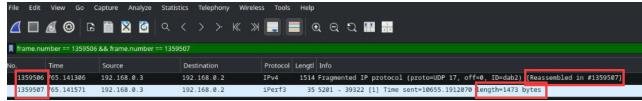


Figure 7: Wireshark capture showing IP fragmentation of a UDP datagram with payload size of 1473 B in the Wi-Fi-to-Wi-Fi scenario (S3).

The values of the goodput for each datagram payload size chosen for the tests are reported in Table 11 and Table 12. Figures 6 and 7 illustrate the effect of IP fragmentation for a UDP payload of 1473 B, which exceeds fragmentation threshold and results in a sharp goodput collapse in the Wi-Fi-to-Wi-Fi scenario (S3).

Table 11: Average goodput for each MTU size - TCP protocol.

MTU size	Goodput
256	3.3656
512	5.7455
768	7.8635
1024	9.3104
1280	10.6733
1500	11.5331

Table 12: Average goodput for each datagram payload size - UDP protocol.

Datagram size	Goodput
16	0.3378
64	1.252
128	2.3809
256	4.27
512	7.1736
1024	10.9385
1472	13.2726
1473	7.2197
2048	6.4997
2944	6.3702

5 CONCLUSION

Full-duplex Ethernet (S1) achieved the highest goodput in both TCP and UDP tests, approaching the theoretical header-efficiency ceiling. Among the wireless configurations, relative performance was strictly dependent on the transport protocol. The single-hop topology (S2) outperformed the double-hop setup (S3) only during TCP tests. For TCP, larger MTUs consistently improved goodput in every scenario by reducing per-frame header cost, though gains diminished above 1280 B on wireless links where larger frames incur higher collision penalties. Conversely, when using the UDP protocol, S2's performance was slightly worse than S3's. For UDP, the fragmentation boundary at 1472 B was the dominant factor: all wireless scenarios exhibited a sharp goodput collapse immediately beyond it, but unexpectedly, this degradation was marginally more pronounced in the single-hop topology than in the double-hop setup. Comparing S1 with S2 isolates the baseline cost of introducing a single wireless hop, while comparing S2 with S3 reveals that the traditional penalty of Wi-Fi-to-Wi-Fi forwarding—where traffic must contend for the wireless medium twice—is highly protocol-dependent. Together, these three scenarios confirm that CSMA/CA overhead, IP fragmentation, and external interference are the three principal impairments to IEEE 802.11g goodput, but their interactive effects vary significantly between reliable (TCP) and stateless (UDP) transmissions.

APPENDIX

The following script was used for automated tests and data collection in CSV format.

```
#!/usr/bin/env python3

import argparse
import subprocess
import json
import csv
import sys
import time
import os
import statistics
import re
import signal
from datetime import datetime

DEFAULT_REPETITIONS = 10
DEFAULT_DURATION = 10
DEFAULT_SLEEP = 1.0
DEFAULT_PORT = 5201
DEFAULT_INTERFACE = "eno1"
DEFAULT_UDP_BITRATE = "40M"
TCP_SIZES = [128, 256, 512, 768, 1024, 1280, 1500]
UDP_SIZES = [16, 64, 128, 256, 512, 1024, 1472, 1473, 2048, 2944]

def parse_args():
    parser = argparse.ArgumentParser(
        description="Automated iperf3 goodput measurements for the WiFi lab.",
    )
    parser.add_argument("--protocol", "-p",
        choices=["tcp", "udp"], required=True)
    parser.add_argument("--server", "-s",
        required=True)
    parser.add_argument("--port",
        type=int,
        default=DEFAULT_PORT)
    parser.add_argument("--repetitions",
        type=int,
        default=DEFAULT_REPETITIONS)
    parser.add_argument("--duration",
        type=int,
        default=DEFAULT_DURATION)
    parser.add_argument("--sleep",
        type=float,
        default=DEFAULT_SLEEP)
    parser.add_argument("--reverse", "-R", action=
        "store_true")
    parser.add_argument("--udp-bitrate", "-b",
        default=DEFAULT_UDP_BITRATE, metavar="
        BITRATE")
    size_group = parser.
        add_mutually_exclusive_group()
    size_group.add_argument("--sizes",
        type=
        int, nargs="+", metavar="BYTES")
    size_group.add_argument("--size-range", nargs
        =3, type=int, metavar=("START", "STOP",
        "STEP"))
    parser.add_argument("--interface", "-i",
        default=DEFAULT_INTERFACE)
```

```
parser.add_argument("--scenario", "-S",
    default="default")
parser.add_argument("--output", "-o",
    default=None)
return parser.parse_args()

def get_mtu(interface: str) -> int | None:
    result = subprocess.run(["ip", "link", "show",
        interface], capture_output=True, text=
        True)
    match = re.search(r"\bmtu\s+(\d+)", result.
        stdout)
    return int(match.group(1)) if match else None

def set_mtu(interface: str, mtu: int) -> bool:
    result = subprocess.run(
        ["sudo", "ip", "link", "set", interface, "
        mtu", str(mtu)],
        capture_output=True, text=True,
    )
    return result.returncode == 0

def run_iperf3(
    server: str,
    port: int,
    protocol: str,
    payload_size: int,
    duration: int,
    reverse: bool,
    udp_bitrate: str,
) -> float | None:
    cmd = ["iperf3", "-c", server, "-p", str(port)
        , "-t", str(duration), "-J"]
    if protocol == "udp":
        cmd += ["-u", "-l", str(payload_size), "-b
            ", udp_bitrate]
    if reverse:
        cmd.append("-R")

    try:
        result = subprocess.run(cmd,
            capture_output=True, text=True,
            timeout=duration + 30)
    except subprocess.TimeoutExpired:
        return None

    if result.returncode != 0:
        return None

    try:
        data = json.loads(result.stdout)
    except json.JSONDecodeError:
        return None

    if "error" in data:
        return None

    try:
        if protocol == "tcp":
```

```

337         bps = data["end"]["sum_received"]["
338             bits_per_second"]
339     else:
340         # end.sum["bits_per_second"] is the
341         # sender rate; scale by received
342         # packet fraction to get receiver
343         # goodput. Clamp received to [0,
344         # total]
345         # to guard against negative
346         # lost_packets from IP fragmentation
347         .
348         bps_sent = data["end"]["sum"]["
349             bits_per_second"]
350         lost = data["end"]["sum"].get("
351             lost_packets", 0)
352         total = data["end"]["sum"].get("
353             packets", 1)
354         received = min(max(total - lost, 0),
355             total)
356         bps = bps_sent * (received /
357             total) if total > 0 else 0.0
358     except KeyError:
359         return None
360
361     return bps / 1e6
362
363 def compute_stats(values: list[float]) -> dict:
364     n = len(values)
365     return {
366         "avg_mbps": statistics.mean(
367             values),
368         "min_mbps": min(values),
369         "max_mbps": max(values),
370         "std_mbps": statistics.stdev(
371             values) if n > 1 else 0.0,
372         "n_successful_runs": n,
373     }
374
375 def main():
376     args = parse_args()
377
378     if args.sizes:
379         sizes = sorted(set(args.sizes))
380     elif args.size_range:
381         start, stop, step = args.size_range
382         sizes = list(range(start, stop + 1, step))
383         if stop not in sizes:
384             sizes.append(stop)
385     else:
386         sizes = TCP_SIZES if args.protocol == "tcp"
387         " else UDP_SIZES
388
389     if not sizes:
390         sys.exit(1)
391
392     timestamp = datetime.now().strftime("%Y%m%d_
393         %H%M%S")
394     direction = "downstream" if args.reverse
395     else "upstream"

```

```

398     output_file = args.output or f"results_{args.
399         scenario}_{args.protocol}_{direction}_{
400         timestamp}.csv"
401
402     original_mtu = None
403     if args.protocol == "tcp":
404         original_mtu = get_mtu(args.interface)
405         if original_mtu is None:
406             sys.exit(1)
407
408     interrupted = False
409
410     def _restore_and_exit(sig, frame):
411         nonlocal interrupted
412         interrupted = True
413
414     signal.signal(signal.SIGINT,
415         _restore_and_exit)
416     signal.signal(signal.SIGTERM,
417         _restore_and_exit)
418
419     all_results = []
420     size_stats = []
421
422     try:
423         for size in sizes:
424             if interrupted:
425                 break
426
427             if args.protocol == "tcp":
428                 if not set_mtu(args.interface,
429                     size):
430                     continue
431                 time.sleep(0.5)
432
433             run_goodputs = []
434
435             for rep in range(1, args.repetitions +
436                 1):
437                 if interrupted:
438                     break
439
440                 gput = run_iperf3(
441                     server=args.server,
442                     port=args.port,
443                     protocol=args.protocol,
444                     payload_size=size,
445                     duration=args.duration,
446                     reverse=args.reverse,
447                     udp_bitrate=args.udp_bitrate,
448                 )
449
450                 fragmented = (args.protocol == "
451                     tcp" and size > 1460) or \
452                     (args.protocol == "
453                     udp" and size >
454                     1472)
455
456                 if gput is not None:
457                     run_goodputs.append(gput)
458                     status, goodput_val = "OK",
459                     gput

```

```

460         else:
461             status, goodput_val = "FAIL",
462                 0.0
463
464         all_results.append({
465             "scenario":    args.scenario,
466             "protocol":    args.protocol,
467             "direction":   direction,
468             "frame_size_b": size,
469             "fragmented":  "Y" if
470                 fragmented else "N",
471             "iteration":    rep,
472             "goodput_mbps": goodput_val,
473             "status":       status,
474         })
475
476         if rep < args.repetitions and not
477             interrupted:
478             time.sleep(args.sleep)
479
480     if run_goodputs:
481         stats = compute_stats(run_goodputs
482             )
483         _frag = (args.protocol == "tcp"
484             and size > 1460) or \
485             (args.protocol == "udp"
486                 and size > 1472)
487         size_stats.append({
488             "scenario":    args.scenario,
489             "protocol":    args.protocol,
490             "direction":   direction,
491             "frame_size_b": size,
492             "fragmented":  "Y" if _frag
493                 else "N",
494             "iteration":    "summary",
495             "goodput_mbps": "",
496             "status":       f"{stats['
497                 n_successful_runs']/args
498                 .repetitions}",
499             "avg_mbps":     f"{stats['
500                 avg_mbps']:}.4f)",
501             "min_mbps":     f"{stats['
502                 min_mbps']:}.4f)",
503             "max_mbps":     f"{stats['
504                 max_mbps']:}.4f)",
505             "std_mbps":     f"{stats['
506                 std_mbps']:}.4f)",
507         })
508
509     finally:
510         if args.protocol == "tcp" and original_mtu
511             is not None:
512             set_mtu(args.interface, original_mtu)
513
514     if all_results:
515         fieldnames = [
516             "scenario", "protocol", "direction
517             ", "frame_size_b",
518             "fragmented", "iteration", "
519             goodput_mbps", "status",
520             "avg_mbps", "min_mbps", "max_mbps"
521             , "std_mbps",

```

```

522     ]
523     with open(output_file, "w", newline="")
524         ) as f:
525         writer = csv.DictWriter(f,
526             fieldnames=fieldnames)
527         writer.writeheader()
528         writer.writerows(
529             {**{k: (f"{v:.4f}" if
530                 isinstance(v, float) else
531                 v)
532                 for k, v in row.items()}},
533             "avg_mbps": "", "min_mbps": "
534             ", "max_mbps": "", "
535             std_mbps": ""}
536             for row in all_results
537         )
538         writer.writerows(size_stats)
539
540         if interrupted:
541             sys.exit(130)
542
543     if __name__ == "__main__":
544         main()
545

```

REFERENCES

- [1] Jangeun Jun, P. Peddabachagari, and M. Sichitiu. 2003. Theoretical maximum throughput of IEEE 802.11 and its applications. In *Second IEEE International Symposium on Network Computing and Applications, 2003. NCA 2003*. 249–256. <https://doi.org/10.1109/NCA.2003.1201163>